

Laporan Praktikum 11 — Queue (Antrean)

Mata Kuliah: Struktur Data

Nama: Muhammad Said Habibi

NIM: 25030101968

Topik: Queue (Antrean)

Daftar Isi

1. Analisa Perbedaan Program 1 dan Program 2
 2. Contoh Program Queue dengan Linked List
 3. Contoh Program Penerapan Queue pada Aplikasi
-

Tugas 1 — Analisa Perbedaan Program 1 dan Program 2

1.1 Program 1 — Linear Queue (Queue2.java)

Program 1 menggunakan implementasi **Linear Queue** berbasis array.

Queue2.java

```
public class Queue2{
    private int[] arr;
    private int rear;
    private int capacity;

    public Queue2(int size) {
        capacity = size;
        arr = new int[capacity];
        rear = -1;
    }

    public boolean isFull() {
        return rear == capacity - 1;
    }

    public boolean isEmpty() {
        return rear == -1;
    }

    public void enqueue (int data) {
        if (isFull()) {
            System.out.println("Antrean penuh");
        } else {
            arr[++rear] = data;
            System.out.println(data + " masuk antrean");
        }
    }

    public int dequeue() {
        if(isEmpty()) {
            System.out.println("Antrean kosong");
            return -1;
        }
        int frontElement = arr[0];
        for (int i = 0; i < rear; i++){
            arr[i] = arr[i + 1];
        }
        rear--;
        return frontElement;
    }

    public int size(){
        return rear + 1;
    }

    public void display() {
        if (isEmpty()) {
            System.out.println("Antrean kosong");
            return;
        }
        System.out.println("isi antrean: ");
        for (int i = 0; i <= rear; i++){
            System.out.println(arr[i] + " ");
        }
    }
}
```

```

    }
    System.out.println();
}

public void peek(){
    if (isEmpty()){
        System.out.println("Antrean kosong");
    }
    System.out.println("Elemen depan: "+arr[0]);
}
}

```

Main.java (Program 1)

```

public class Main {
    public static void main(String[] args) {
        Queue2 q = new Queue2(3);
        q.enqueue(5);
        q.enqueue(10);
        q.enqueue(15);
        q.display();
        System.out.println("Data yang keluar " + q.dequeue());
        q.display();
        q.peek();
    }
}

```

Output Program 1:

```

said@said-mint:~/BACKUP/GitHub/belajar-struktur-data/Praktikum11$ cd /home/said/BACKUP/GitHub/belajar-struktur-data/Praktikum11 ; /usr/bin/env /usr/lib/jvm/java-17-openjdk-amd64/bin/java -XX:+ShowCodeDetailsInExceptionMessages -cp /home/said/.config/Code/User/workspaceStorage/3025904b0affaacea0018447e5cacf27/redhat.java/jdt_ws/Praktikum11_7d60b072/bin Main
5 masuk antrean
10 masuk antrean
15 masuk antrean
isi antrean:
5
10
15

Data yang keluar 5
isi antrean:
10
15

Elemen depan: 10
said@said-mint:~/BACKUP/GitHub/belajar-struktur-data/Praktikum11$

```

1.2 Program 2 — Circular Queue (Queue.java)

Program 2 menggunakan implementasi **Circular Queue** berbasis array.

Queue.java

```
package Program2;

public class Queue {
    private int[] arr;
    private int front;
    private int rear;
    private int capacity;
    private int count;

    public Queue(int size) {
        arr = new int[size];
        capacity = size;
        front = 0;
        rear = -1;
        count = 0;
    }

    public void enqueue(int data) {
        if (isFull()){
            System.out.println("Overflow! Antrean");
            return;
        }
        rear = (rear + 1) % capacity;
        arr[rear] = data;
        count++;
        System.out.println("Dimasukan: " + data);
    }

    public int dequeue(){
        if (isEmpty()){
            System.out.println("Underflow! antrean kosong");
            return -1;
        }
        int x = arr[front];
        front = (front + 1) % capacity;
        count--;
        return x;
    }

    public int getFront() {
        if (isEmpty()) return -1;
        return arr[front];
    }

    public int getRear () {
        if (isEmpty()) return -1;
        return arr[rear];
    }

    public int size() {
        return count;
    }
}
```

```

    public boolean isEmpty(){
        return (count == 0);
    }

    public boolean isFull(){
        return (count == capacity);
    }
}

```

Main.java (Program 2)

```

package Program2;

public class Main {
    public static void main(String[] args) {
        Queue q = new Queue(5);
        q.enqueue(10);
        q.enqueue(20);
        q.enqueue(30);
        System.out.println("Element depan: " + q.getFront());
        System.out.println("Element belakang: " + q.getRear());
        System.out.println("Ukuran antrean: " + q.size());
        System.out.println("Dikeluarkan: " + q.dequeue());
        System.out.println("apakah kosong? " + q.isEmpty());
    }
}

```

Output Program 2:

```

said@said-mint:~/BACKUP/GitHub/belajar-struktur-data/Praktikum11$ cd /home/said/BACKUP/GitHub/belajar-struktur-data/Praktikum11 ; /usr/bin/env /usr/lib/jvm/java-17-openjdk-amd64/bin/java -XX:+ShowCodeDetailsInExceptionMessages -cp /home/said/.config/Code/User/workspaceStorage/3025904b0affaacea0018447e5cacf27/redhat.java/jdt_ws/Praktikum11_7d60b072/bin Program2.Main
Dimasukan: 10
Dimasukan: 20
Dimasukan: 30
Element depan: 10
Element belakang: 30
Ukuran antrean: 3
Dikeluarkan: 10
apakah kosong? false
said@said-mint:~/BACKUP/GitHub/belajar-struktur-data/Praktikum11$

```

1.3 Analisa Perbedaan

No	Aspek	Program 1 (Linear Queue)	Program 2 (Circular Queue)
1	Jenis Queue	Linear Queue	Circular Queue
2	Variabel penunjuk	Hanya rear	front, rear, dan count
3	Cara kerja enqueue	<code>arr[++rear] = data</code>	<code>rear = (rear + 1) % capacity</code>
4	Cara kerja dequeue	Ambil elemen indeks 0, lalu geser seluruh elemen ke kiri	Ambil elemen pada posisi front, lalu majukan front dengan modulo
5	Kompleksitas dequeue	$O(n)$ — harus geser semua elemen	$O(1)$ — hanya ubah indeks front
6	Cek isEmpty	<code>rear == -1</code>	<code>count == 0</code>
7	Cek isFull	<code>rear == capacity - 1</code>	<code>count == capacity</code>
8	Efisiensi memori	Kurang efisien — ruang di depan array tidak bisa dipakai ulang	Lebih efisien — ruang yang sudah di-dequeue bisa dipakai kembali
9	Fitur tambahan	<code>peek()</code> , <code>display()</code>	<code>getFront()</code> , <code>getRear()</code>

Penjelasan Perbedaan Utama:

1. Mekanisme Dequeue

Pada Program 1, ketika elemen dikeluarkan, seluruh elemen tersisa harus digeser satu posisi ke kiri dengan perulangan. Proses ini memakan waktu $O(n)$. Pada Program 2, dequeue hanya mengubah nilai indeks front menggunakan operasi modulo sehingga hanya memakan waktu $O(1)$.

2. Konsep Sirkular (Modulo)

Program 2 menggunakan operasi modulo (%) untuk membuat array "berputar". Ketika rear atau front mencapai ujung array, ia akan kembali ke indeks 0, sehingga ruang yang sudah kosong bisa dipakai kembali.

3. Pengelolaan Ukuran

Program 1 mengandalkan rear saja untuk mengetahui jumlah elemen. Program 2 menggunakan variabel count yang terpisah karena pada circular queue, rear bisa bernilai lebih kecil dari front.

Kesimpulan: Program 2 (Circular Queue) lebih efisien karena operasi dequeue $O(1)$ dan pemanfaatan memori yang lebih optimal. Program 1 (Linear Queue) lebih sederhana dan cocok untuk pembelajaran dasar.

Tugas 2 — Contoh Program Queue dengan Linked List

Berikut adalah implementasi Queue menggunakan Linked List. Berbeda dengan array, Linked List tidak memiliki batasan ukuran tetap dan tidak memerlukan pergeseran elemen saat dequeue.

2.1 Kode Program

Node.java

```
package Tugas.QueueLinkedList;

public class Node {
    int data;
    Node next;

    public Node(int data){
        this.data = data;
        this.next = null;
    }
}
```


QueueLL.java

```
package Tugas.QueueLinkedList;

public class QueueLL {
    private Node front;
    private Node rear;
    private int size;

    public QueueLL(){
        front = null;
        rear = null;
        size = 0;
    }

    public void enqueue(int data){
        Node newNode = new Node(data);
        if (isEmpty()){
            front = newNode;
            rear = newNode;
        }else{
            rear.next = newNode;
            rear = newNode;
        }
        size++;
        System.out.println("Dimasukan: " + data);
    }

    public int dequeue() {
        if(isEmpty()){
            System.out.println("Underflow! Antrean kosong");
            return -1;
        }
        int removed = front.data;
        front = front.next;
        if (front == null){
            rear = null;
        }
        size--;
        return removed;
    }

    public int peek(){
        if(isEmpty()){
            System.out.println("Antrean kosong");
            return -1;
        }
        return front.data;
    }

    public boolean isEmpty() {
        return front == null;
    }

    public int size() {
```

```

        return size;
    }

    public void display(){
        if (isEmpty()){
            System.out.println("Antrean kosong");
            return;
        }
        System.out.print("isi antrean (front -> rear): ");
        Node current = front;
        while (current != null) {
            System.out.print(current.data);
            if (current.next != null) System.out.print(" -> ");
            current = current.next;
        }
        System.out.println();
    }
}

```

Main.java

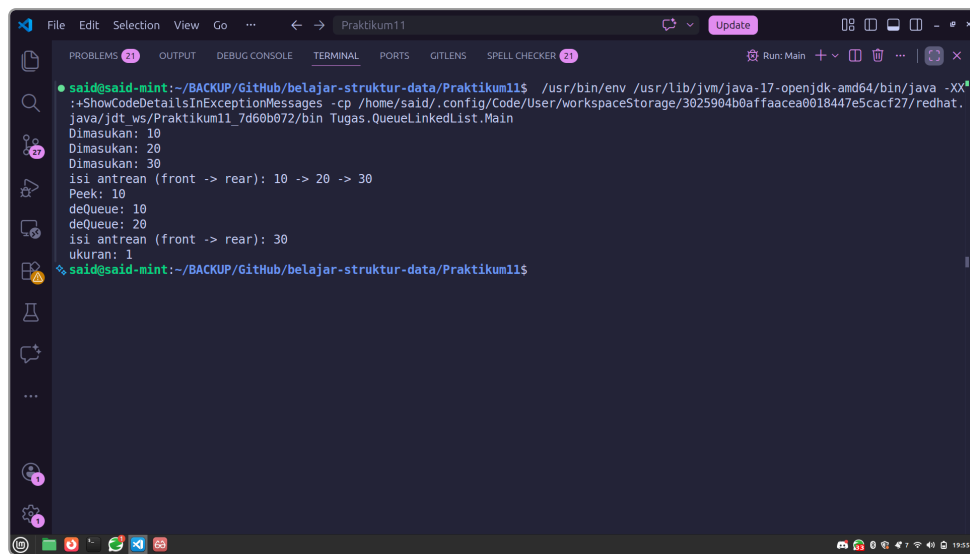
```

package Tugas.QueueLinkedList;

public class Main {
    public static void main(String[] args) {
        QueueLL q = new QueueLL();
        q.enqueue(10);
        q.enqueue(20);
        q.enqueue(30);
        q.display();
        System.out.println("Peek: " + q.peek());
        System.out.println("deQueue: " + q.dequeue());
        System.out.println("deQueue: " + q.dequeue());
        q.display();
        System.out.println("ukuran: " + q.size());
    }
}

```

2.2 Output Program



```
Praktikum11
sai@sai-mint:~/BACKUP/GitHub/belajar-struktur-data/Praktikum11$ ./usr/bin/env /usr/lib/jvm/java-17-openjdk-amd64/bin/java -XX*
:~ShowCodeDetailsInExceptionMessages -cp /home/sai/.config/Code/User/workspaceStorage/3025904b0affaacea0018447e5cacf27/redhat.
java/jdt_ws/Praktikum11_7d60b072/bin Tugas.QueueLinkedList.Main
Dimasukan: 10
Dimasukan: 20
Dimasukan: 30
isi antrian (front -> rear): 10 -> 20 -> 30
Peek: 10
deQueue: 10
deQueue: 20
isi antrian (front -> rear): 30
ukuran: 1
sai@sai-mint:~/BACKUP/GitHub/belajar-struktur-data/Praktikum11$
```

2.3 Penjelasan

Setiap elemen dalam queue direpresentasikan sebagai Node yang saling terhubung melalui pointer next.

Operasi enqueue: Buat node baru, jika kosong maka front dan rear menunjuk ke node baru. Jika tidak, sambungkan di belakang rear lalu pindahkan rear ke node baru.

Operasi dequeue: Simpan data dari front, pindahkan front ke node berikutnya. Jika front menjadi null, rear juga di-set null. Node lama otomatis di-garbage collect oleh Java.

Aspek	Queue Array	Queue Linked List
Ukuran	Tetap (fixed)	Dinamis (fleksibel)
Dequeue	Perlu geser elemen atau modulo	Cukup pindah pointer front — $O(1)$
Memori	Dialokasikan sekaligus di awal	Dialokasikan per node sesuai kebutuhan
Overflow	Bisa terjadi jika array penuh	Tidak terjadi selama memori tersedia

Tugas 3 — Contoh Program Penerapan Queue pada Aplikasi

3.1 Tema: Simulasi Antrean Genshin Impact Cloud Gaming

Program ini mensimulasikan sistem antrean pada layanan Genshin Impact Cloud Gaming. Ketika banyak pemain ingin bermain melalui layanan cloud, mereka harus mengantre untuk mendapatkan slot server. Konsep ini sesuai dengan prinsip **FIFO (First In, First Out)** pada Queue.

3.2 Kode Program

GenshinCloudQueue.java

```
package Tugas.QueueGenshinImpactCloud;

public class GenshinCloudQueue {
    private String[] queue;
    private int rear;
    private int capacity;

    public GenshinCloudQueue(int size) {
        this.capacity = size;
        this.queue = new String[capacity];
        this.rear = -1;
    }

    public boolean isFull() {
        return rear == capacity - 1;
    }

    public boolean isEmpty() {
        return rear == -1;
    }

    public void enqueue(String playerName) {
        if (isFull()) {
            System.out.println("\n[!] Antrean Penuh! Server sedang sibuk.");
        } else {
            queue[++rear] = playerName;
            System.out.println("\n[+] " + playerName
                + " telah masuk ke dalam antrean.");
            System.out.println("Posisi saat ini: " + (rear + 1));
        }
    }

    public String dequeue() {
        if (isEmpty()) {
            System.out.println("\n[!] Antrean Kosong.");
            return null;
        }
        String playingPlayer = queue[0];
        for (int i = 0; i < rear; i++) {
            queue[i] = queue[i + 1];
        }
        rear--;
        System.out.println("\n[>>] " + playingPlayer
            + " sekarang bisa mulai bermain!");
        return playingPlayer;
    }

    public void displayQueue() {
        if (isEmpty()) {
            System.out.println("\n[i] Antrean saat ini kosong.");
        }
    }
}
```

```
        return;
    }
    System.out.println(
        "\n--- DAFTAR ANTREAN GENSHIN CLOUD ---");
    for (int i = 0; i <= rear; i++) {
        System.out.println((i + 1) + ". " + queue[i]);
    }
    System.out.println(
        "-----");
}

public int getQueueCount() {
    return rear + 1;
}

public String peekNext() {
    if (isEmpty()) return "Kosong";
    return queue[0];
}
}
```

MainApp.java

```
package Tugas.QueueGenshinImpactCloud;

public class MainApp {
    public static void main(String[] args) {
        GenshinCloudQueue cloudQueue =
            new GenshinCloudQueue(10);

        System.out.println("=====");
        System.out.println(" SIMULASI ANTREAN GENSHIN");
        System.out.println("=====");

        System.out.println(
            "\n--- Menambahkan Pemain ---");
        cloudQueue.enqueue("Traveler_Lumine");
        cloudQueue.enqueue("Diluc_Master");
        cloudQueue.enqueue("Paimon_EmergencyFood");
        cloudQueue.enqueue("Zhongli_GeoDaddy");
        cloudQueue.enqueue("Raiden_Ei");

        cloudQueue.displayQueue();

        System.out.println(
            "\n--- Memproses Antrean ---");
        cloudQueue.dequeue();
        cloudQueue.dequeue();

        System.out.println("Pemain berikutnya: "
            + cloudQueue.peekNext());
        System.out.println("Total menunggu: "
            + cloudQueue.getQueueCount());

        cloudQueue.displayQueue();
        System.out.println("\nSimulasi selesai.");
    }
}
```

3.3 Output Program

```
=====
SIMULASI ANTREAN GENSHIN CLOUD
=====

--- Menambahkan Pemain ke Antrean ---

[+] Traveler_Lumine telah masuk ke dalam antrean.
Posisi saat ini: 1

[+] Diluc_Master telah masuk ke dalam antrean.
Posisi saat ini: 2

[+] Paimon_EmergencyFood telah masuk ke dalam antrean.
Posisi saat ini: 3

[+] Zhongli_GeoDaddy telah masuk ke dalam antrean.
Posisi saat ini: 4

[+] Raiden_Ei telah masuk ke dalam antrean.
Posisi saat ini: 5

--- DAFTAR ANTREAN GENSHIN CLOUD ---
1. Traveler_Lumine
2. Diluc_Master
3. Paimon_EmergencyFood
4. Zhongli_GeoDaddy
5. Raiden_Ei
-----

--- Memproses Antrean (Mulai Bermain) ---

[>>] Traveler_Lumine sekarang bisa mulai bermain!

[>>] Diluc_Master sekarang bisa mulai bermain!

Pemain berikutnya: Paimon_EmergencyFood
Total menunggu: 3

--- DAFTAR ANTREAN GENSHIN CLOUD ---
1. Paimon_EmergencyFood
2. Zhongli_GeoDaddy
3. Raiden_Ei
-----

Simulasi selesai.
```


3.4 Penjelasan

Konsep Queue	Penerapan di Genshin Cloud
FIFO	Pemain yang pertama masuk antrean akan pertama mendapat akses bermain
Enqueue	Pemain bergabung ke antrean cloud gaming
Dequeue	Pemain di posisi depan mendapat giliran bermain
Peek	Melihat siapa pemain selanjutnya yang akan bermain
isFull	Server sudah penuh, tidak bisa menampung pemain lagi
isEmpty	Tidak ada pemain yang mengantre

Queue cocok untuk kasus ini karena layanan cloud gaming memiliki keterbatasan server, dan pemain yang sudah menunggu lebih lama harus mendapat giliran lebih dahulu (prinsip keadilan FIFO).

Kesimpulan

1. **Queue** adalah struktur data yang mengikuti prinsip FIFO (First In, First Out).
2. **Linear Queue** lebih sederhana namun dequeue-nya $O(n)$. **Circular Queue** lebih efisien dengan dequeue $O(1)$.
3. **Queue Linked List** memberikan fleksibilitas ukuran dinamis dengan operasi enqueue dan dequeue $O(1)$.
4. Queue banyak diterapkan di kehidupan nyata, seperti sistem antrean cloud gaming.